Mini-Project 2

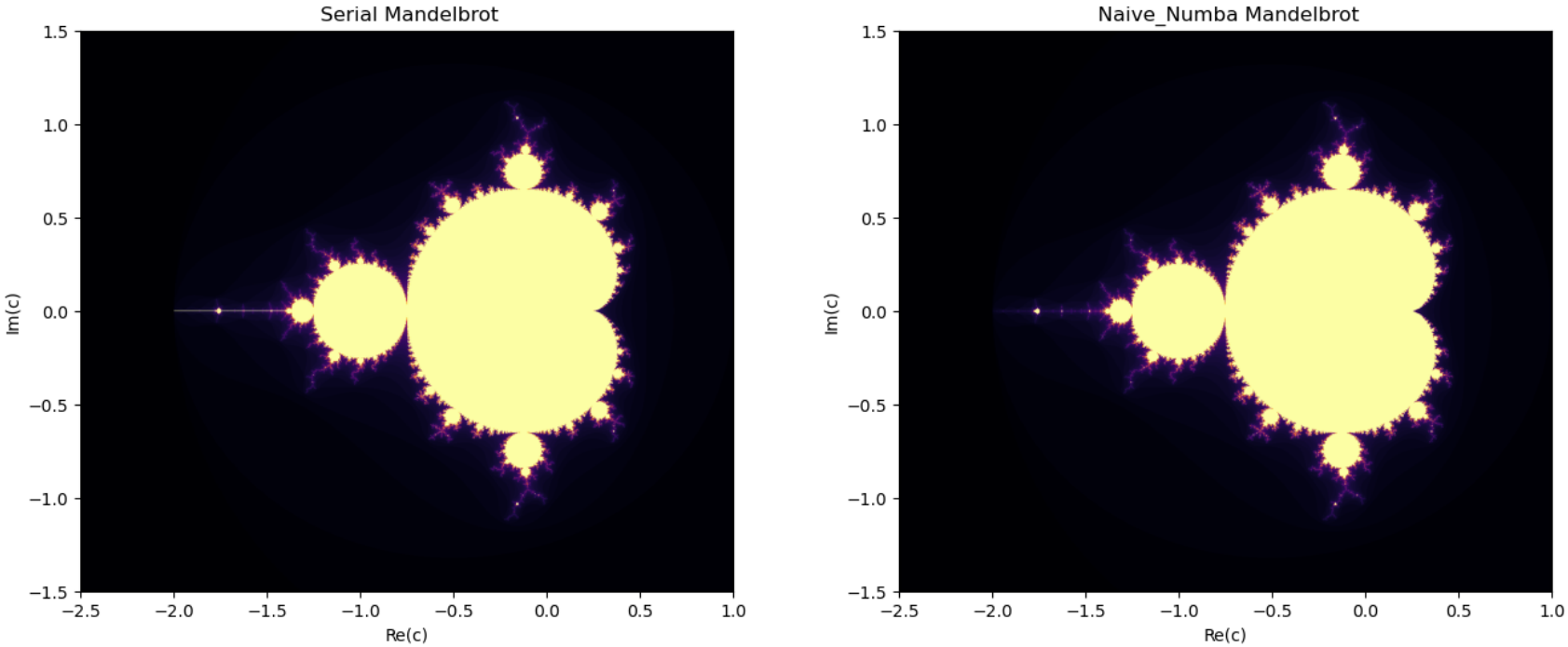
Section 1 - Multiprocessing (Results from: L04 (M1-M2-M3), L05 (M1-M2–M3))

Lecture 4 - Milestone 1

In [1]:
```
import numpy as np
from numba import njit
from multiprocessing import Pool
import time, os, statistics, matplotlib.pyplot as plt
import sys
from pathlib import Path

# Adds mandelbrot-nsc to sys.path so lecture_files can be imported
sys.path.append(str(Path("..").resolve()))

import lecture_py_files.mandelbrot_3 as mb3
import lecture_py_files.mandelbrot_4 as mb4

# Parameters
iterations = 100
N = 1024
x_dim = (-2.5, 1.0)
y_dim = (-1.5, 1.5)
resolution = (1024, 1024)

#Numba and serial compared
result_full = mb3.compute_mandelbrot_full(x_dim=x_dim, y_dim=y_dim, res=resolution, max_iter=iterations)
result_serial = mb4.mandelbrot_serial(N=N, x_dim=x_dim, y_dim=y_dim, max_iter=iterations)

fig, axes = plt.subplots(1, 2, figsize=(16,6))
axes[0].imshow(result_serial, extent=[x_dim[0], x_dim[1], y_dim[0], y_dim[1]], cmap='inferno', origin='lower', aspect
axes[0].set_title(' Serial Mandelbrot')
axes[0].set_xlabel('Re(c)')
axes[0].set_ylabel('Im(c)')
```

```
axes[1].imshow(result_full, extent=[x_dim[0], x_dim[1], y_dim[0], y_dim[1]], cmap='inferno', origin='lower', aspect='
axes[1].set_title(' Naive_Numba Mandelbrot')
axes[1].set_xlabel('Re(c)')
axes[1].set_ylabel('Im(c)')
```

Out[1]: Text(0, 0.5, 'Im(c)')



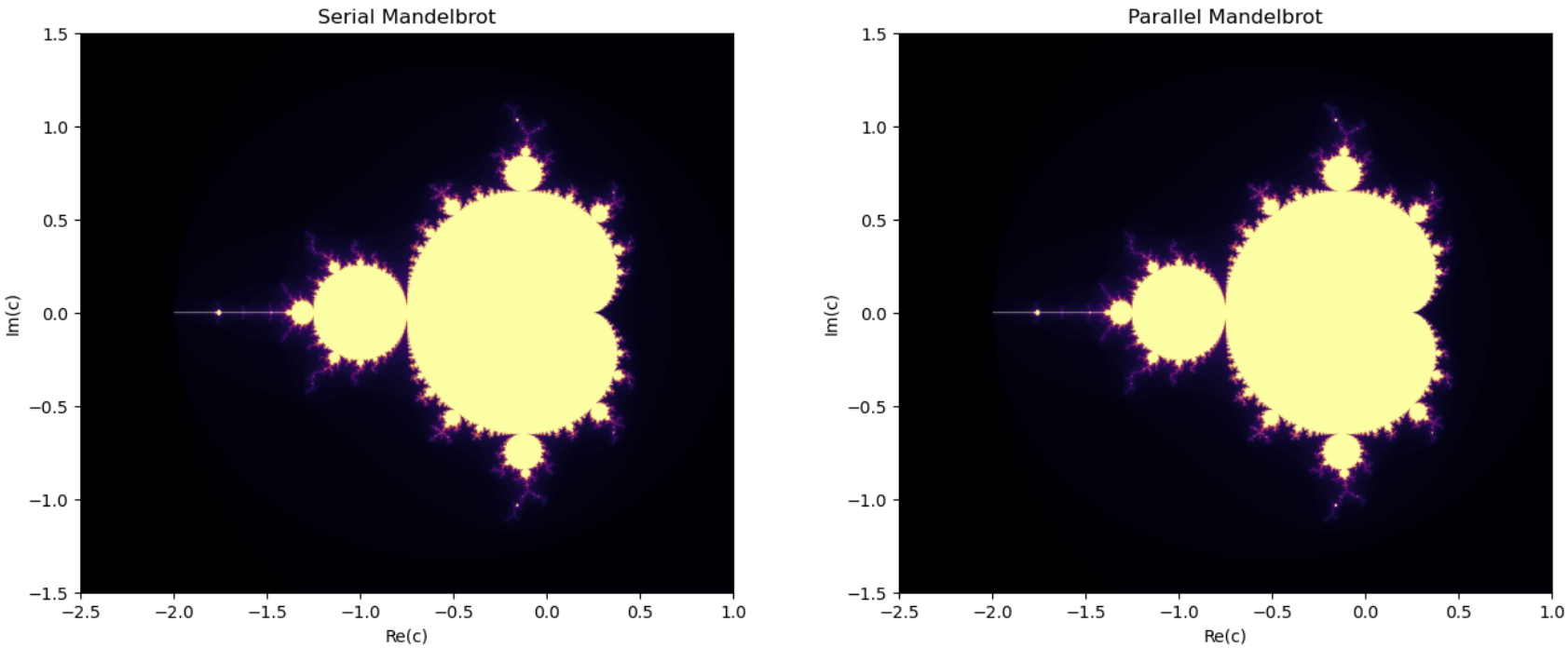
The results of the serial mandelbrot implementation (Lecture 4) matches the naive numba mandelbrot implementation (Lecture 3).

Lecture 4 - Milestone 2

In [2]:
```
# Parameters
n_workers = 4

# Milestone 2 testing:
result_serial = mb4.mandelbrot_serial(N=N, x_dim=x_dim, y_dim=y_dim)
result_parallel = mb4.mandelbrot_parallel(N=N, x_dim=x_dim, y_dim=y_dim, num_workers=n_workers)
```

```
fig, axes = plt.subplots(1, 2, figsize=(16,6))
axes[0].imshow(result_serial, extent=[x_dim[0], x_dim[1], y_dim[0], y_dim[1]], cmap='inferno', origin='lower', aspect
axes[0].set_title('Serial Mandelbrot')
axes[0].set_xlabel('Re(c)')
axes[0].set_ylabel('Im(c)')

axes[1].imshow(result_parallel, extent=[x_dim[0], x_dim[1], y_dim[0], y_dim[1]], cmap='inferno', origin='lower', aspe
axes[1].set_title('Parallel Mandelbrot')
axes[1].set_xlabel('Re(c)')
axes[1].set_ylabel('Im(c)')
```

Out[2]: Text(0, 0.5, 'Im(c)')



The results of the parallel mandelbrot implementation (Lecture 4) matches the serial mandelbrot implementation (Lecture 4).

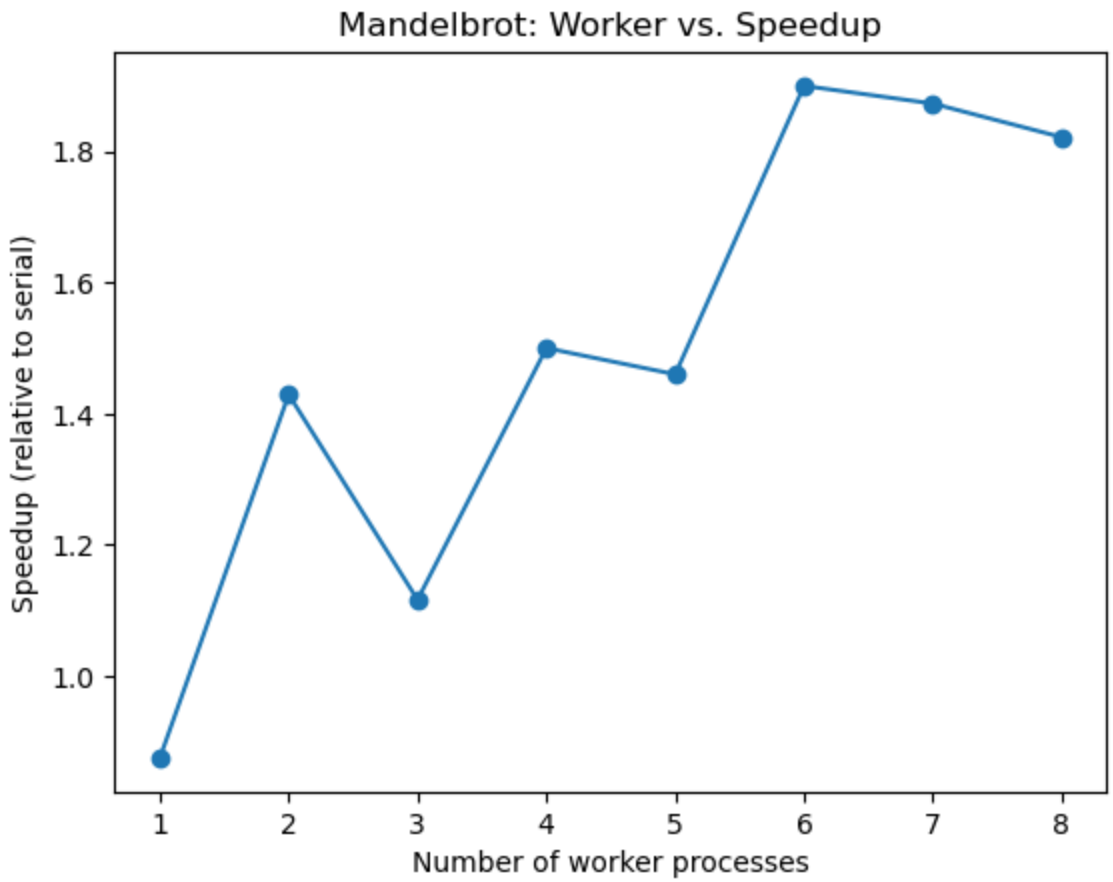
Lecture 4 - Milestone 3

In [3]:
```
import lecture_exercises.monte_carlo_example as mc
# Parameters
```

```python
# N, iterations, dimensions are set in L4-M1
n_runs = 3

workers_list = []
speedups = []

# Serial baseline (Numba already warm after M1 warm-up)
times = []
for _ in range(n_runs):
    start_time = time.perf_counter()
    mb4.mandelbrot_serial(N=N, x_dim=x_dim, y_dim=y_dim, max_iter=iterations)
    times.append(time.perf_counter() - start_time)
t_serial = statistics.median(times)

for n_workers in range(1, os.cpu_count() + 1):
    chunk_size = max(1, N // n_workers)
    chunks = []
    row = 0
    while row < N:
        row_end = min(row + chunk_size, N) # determine end of the row
        chunks.append((row, row_end, N, x_dim, y_dim, iterations))
        row = row_end

    with Pool(processes=n_workers) as pool:
        pool.map(mb4._worker, chunks)
        times = []
        for _ in range(n_runs):
            t0 = time.perf_counter()
            np.vstack(pool.map(mb4._worker, chunks))
            times.append(time.perf_counter() - t0)
    t_parallel = statistics.median(times)
    speedup = t_serial / t_parallel

    # Append workers and speedups for plotting
    workers_list.append(n_workers)
    speedups.append(speedup)

    # print table of workers, speedup and efficiency
    print(f'{n_workers:2d} workers | '
            f'parallel time: {t_parallel:.4f}s | '
            f'speedup: {speedup:.2f}x | '
            f'efficiency: {speedup/n_workers:.2f}')
```

```python
# get peak for computing the implied s
index_peak = np.argmax(speedups)
speedup_peak = speedups[index_peak]        # optimal speedup
n_workers_peak = workers_list[index_peak]  # optimal number of workers
print(f'{speedup_peak=:.2f}, {n_workers_peak=}')

s_4 = (1/speedup_peak - 1/n_workers_peak) / (1 - 1/n_workers_peak)
print(f' Implied s: {s_4:.2f}')

# Plot worker vs. speedup
mc.plot_worker_speedup(title="Mandelbrot", workers=workers_list, speedup=speedups)
plt.savefig("../figures/my_figure.png")
```

```
 1 workers | parallel time: 0.0724s | speedup: 0.88x | efficiency: 0.88
 2 workers | parallel time: 0.0443s | speedup: 1.43x | efficiency: 0.71
 3 workers | parallel time: 0.0568s | speedup: 1.12x | efficiency: 0.37
 4 workers | parallel time: 0.0423s | speedup: 1.50x | efficiency: 0.38
 5 workers | parallel time: 0.0434s | speedup: 1.46x | efficiency: 0.29
 6 workers | parallel time: 0.0334s | speedup: 1.90x | efficiency: 0.32
 7 workers | parallel time: 0.0338s | speedup: 1.87x | efficiency: 0.27
 8 workers | parallel time: 0.0348s | speedup: 1.82x | efficiency: 0.23
speedup_peak=1.90, n_workers_peak=6
 Implied s: 0.43
```




```
<Figure size 640x480 with 0 Axes>
```

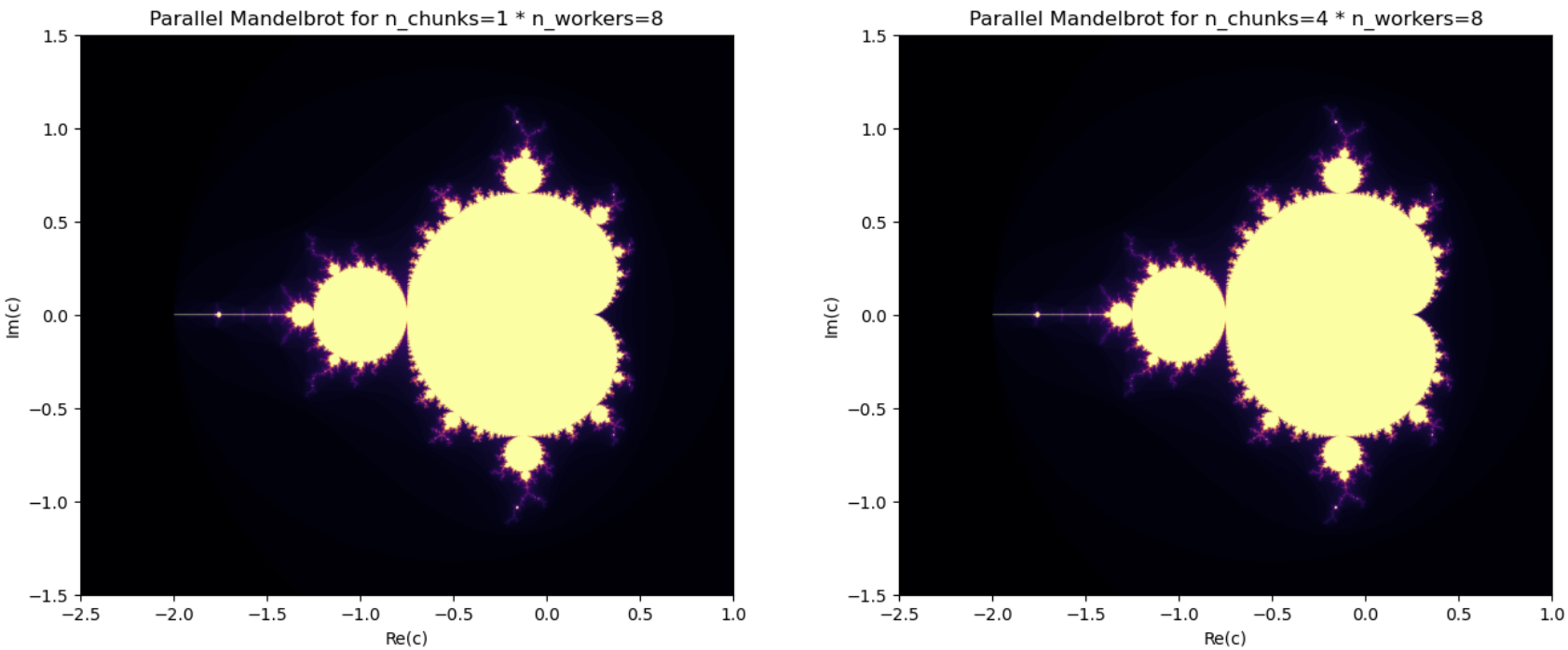
Lecture 5 - Multiprocessing + Dask

Lecture 5 - Milestone 1

Verify result matches serial output for n_chunks

In [4]:
```
import lecture_py_files.mandelbrot_5 as mb5

# Milestone 1 testing:
n_chunks = 4
```

```
result_parallel_1nchunk = mb5.mandelbrot_parallel(N=N, x_dim=x_dim, y_dim=y_dim, num_workers=n_workers, n_chunks=1*n_
result_parallel_4nchunks = mb5.mandelbrot_parallel(N=N, x_dim=x_dim, y_dim=y_dim, num_workers=n_workers, n_chunks=n_c

fig, axes = plt.subplots(1, 2, figsize=(16,6))
axes[0].imshow(result_parallel_1nchunk, extent=[x_dim[0], x_dim[1], y_dim[0], y_dim[1]], cmap='inferno', origin='lowe
axes[0].set_title(f'Parallel Mandelbrot for n_chunks=1 * {n_workers=}')
axes[0].set_xlabel('Re(c)')
axes[0].set_ylabel('Im(c)')

axes[1].imshow(result_parallel_4nchunks, extent=[x_dim[0], x_dim[1], y_dim[0], y_dim[1]], cmap='inferno', origin='low
axes[1].set_title(f'Parallel Mandelbrot for {n_chunks=} * {n_workers=}')
axes[1].set_xlabel('Re(c)')
axes[1].set_ylabel('Im(c)')
plt.show()
```



The parallel mandelbrot for n_chunks=4 and n_workers=8 matches the result of the parallel mandelbrot with n_chunks=1. It also matches the serial mandelbrot from lecture 4 seen in (L4-M2)

Lecture 5 - Milestone 2

Record optimal n_chunks and LIF in performance notebook (MP2)

In [5]:
```
# Parameters
N = 1024
iterations = 100
n_workers = n_workers_peak
print(f'{n_workers_peak=}')
x_dim = (-2.5, 1.0)
y_dim = (-1.25, 1.25)
n_runs = 3

mb5.mandelbrot_chunk(row_start=0, row_end=8, N=8, x_dim=x_dim, y_dim=y_dim, max_iter=iterations) # warm up JIT

# Serial baseline
times = []
for _ in range(n_runs):
    t0 = time.perf_counter()
    mb5.mandelbrot_chunk(row_start=0, row_end=N, N=N, x_dim=x_dim, y_dim=y_dim, max_iter=iterations)
    times.append(time.perf_counter() - t0)
t_serial = statistics.median(times)
print(f'Serial: {t_serial:.3f}s')

print(f"{'Num Chunks':>12} | {'Parallel Time':>12} (s) | {'Speedup':>8} (x) | {'LIF':>8} |")

# Chunk-count sweep (M2): one Pool per config
t_parallel_arr = []
n_chunks_arr = []
tiny = [(0, 8, 8, x_dim, y_dim, iterations)]
for mult in [1, 2, 4, 8, 16]:
    n_chunks = mult * n_workers
    n_chunks_arr.append(n_chunks)
    with Pool(processes=n_workers) as pool:
        pool.map(mb4._worker, tiny)         # warm-up: load JIT cache in workers
        times = []
        for _ in range(n_runs):
            t0 = time.perf_counter()
            mb5.mandelbrot_parallel(N=N, x_dim=x_dim, y_dim=y_dim, max_iter=iterations, num_workers=n_workers, n_chur
            times.append(time.perf_counter() - t0)
    t_parallel = statistics.median(times)
    t_parallel_arr.append(t_parallel)   # create array for getting minimum t_parallel
    lif_parallel = n_workers * t_parallel / t_serial - 1
```

```
    print(f'{n_chunks:12d} | {t_parallel:17.3f} | {t_serial/t_parallel:12.1f} | {lif_parallel:>8.2f} |')

# Determining optimal n_chunks:
min_arg = np.argmin(t_parallel_arr)
opt_n_chunks = n_chunks_arr[min_arg]
opt_t_parallel = t_parallel_arr[min_arg]

print(f'Optimal n_chunks is: {opt_n_chunks}, with time: {opt_t_parallel:.3f}s')
```

```
n_workers_peak=6
Serial: 0.076s
  Num Chunks | Parallel Time (s) |  Speedup (x) |      LIF |
           6 |             0.035 |          2.2 |     1.76 |
          12 |             0.026 |          2.9 |     1.06 |
          24 |             0.024 |          3.1 |     0.94 |
          48 |             0.025 |          3.0 |     0.98 |
          96 |             0.032 |          2.4 |     1.51 |
Optimal n_chunks is: 24, with time: 0.024s
```

Lecture 5 - Milestone 3

Record all results (for all implementations) in performance notebook (MP2) Speedup table:

In [6]:
```
import lecture_py_files.mandelbrot_1_2 as mb12

warmup_res = (64, 64)
resolution = (1024, 1024)
N = 1024
iterations = 100

# 1: Speedup table:
print(f'1. Speedup Table:\n',
      f"{'Implementation':>12} | {'Time (s)':>12} | {'Speedup (x)':>12} |")

# Naive Implementation
times = []
for _ in range(n_runs):
    t0 = time.perf_counter()
    mb12.compute_mandelbrot_naive(x_dim=x_dim, y_dim=y_dim, res=resolution)
    times.append(time.perf_counter() - t0)
```

```python
t_naive = statistics.median(times)
speedup_naive = t_naive/t_naive
print(f"{'Naive Python':>15} | {t_naive:12.3f} | {speedup_naive:12.2f} |")

# Numpy baseline
times = []
for _ in range(n_runs):
    t0 = time.perf_counter()
    mb12.compute_mandelbrot_numpy(x_dim=x_dim, y_dim=y_dim, res=resolution)
    times.append(time.perf_counter() - t0)
t_numpy = statistics.median(times)
speedup_numpy = t_naive/t_numpy
print(f"{'Numpy':>15} | {t_numpy:12.3f} | {speedup_numpy:12.2f} |")

# Numba baseline
times = []
_ = mb3.compute_mandelbrot_full(x_dim=x_dim, y_dim=y_dim, res=warmup_res)  # warmup JIT
for _ in range(n_runs):
    t0 = time.perf_counter()
    mb3.compute_mandelbrot_full(x_dim=x_dim, y_dim=y_dim, res=resolution, max_iter=iterations)
    times.append(time.perf_counter() - t0)
t_numba = statistics.median(times)
speedup_numba = t_naive/t_numba
print(f"{'Numba @njit':>15} | {t_numba:12.3f} | {speedup_numba:12.2f} |")

# Parallel baseline (t_parallel from Milestone 1)
index_t_parallel_peak = np.argmin(t_parallel_arr)
t_parallel = t_parallel_arr[index_t_parallel_peak] # optimal t_parallel
speedup_parallel = t_naive/t_parallel
print(f"{'Parallel (opt)':>15} | {t_parallel:12.3f} | {speedup_parallel:12.2f} |")

# Back-solve implied:
speedup_parallel_numba = t_numba/t_parallel
print(f'{speedup_parallel_numba=}, {n_workers=}')
s_5 = (1/speedup_parallel_numba - 1/n_workers) / (1 - 1/n_workers)

print('\n2. Speedup vs. core count')
print(f'Implied s (L4): {s_4}')
print(f'Implied s (L5): {s_5}')
```

```
1. Speedup Table:
 Implementation |     Time (s) |  Speedup (x) |
   Naive Python |        9.879 |         1.00 |
          Numpy |        2.010 |         4.92 |
    Numba @njit |        0.077 |       128.36 |
 Parallel (opt) |        0.024 |       403.24 |
speedup_parallel_numba=3.14155741302227, n_workers=6

2. Speedup vs. core count
Implied s (L4): 0.43161382135300796
Implied s (L5): 0.18197614820782948
```

3. Brief Recommendation:

Parallel with n_chunks=16 (2x n_workers) results in best speedup at 0.021s giving a speedup of 422x compared to the naive approach. Compared to numba it is approx 3.6x speedup. The Back-solve implied s for L5 is lower than for L4, showing a better balance for the larger number of chunks, while LIF is 1.27 at optimal n_chunks=16

Section 2: Dask Local (Results from: L06 (M1-M2–M3))

Lecture 6 - Milestone 1

In [7]:
```
import lecture_py_files.mandelbrot_6 as mb6
from dask import delayed
from dask.distributed import Client, LocalCluster
import dask, time, statistics

N = 1024
res = (N, N)
max_iter = 100
n_runs = 3

x_dim = (-2.5, 1.0)
y_dim = (-1.25, 1.25)
cluster = LocalCluster(n_workers=8, threads_per_worker=1)
client = Client(cluster)
client.run(lambda: mb5.mandelbrot_chunk(row_start=0, row_end=8, N=8, x_dim=x_dim, y_dim=y_dim, max_iter=max_iter))

times = []
```

```
for _ in range(n_runs):
    t0 = time.perf_counter()
    result_dask = mb6.mandelbrot_dask(N=N, x_dim=x_dim, y_dim=y_dim, max_iter=max_iter)
    times.append(time.perf_counter() - t0)
    time_dask = statistics.median(times)
print(f"Dask local (n_chunks=32): {time_dask:.3f} s")
client.close(); cluster.close()

times = []
for _ in range(n_runs):
    t0 = time.perf_counter()
    mb12.compute_mandelbrot_naive(x_dim=x_dim, y_dim=y_dim, res=res)
    times.append(time.perf_counter() - t0)
    time_naive = statistics.median(times)
print(f'Naive: {time_naive} s')
print(f"Speedup: {time_naive/time_dask:.3f} x")

#Checking whether serial and dask arrays are equal
result_serial = mb4.mandelbrot_serial(N=N, x_dim=x_dim, y_dim=y_dim, max_iter=max_iter)
print(np.array_equal(result_serial, result_dask))
```

```
Dask local (n_chunks=32): 0.224 s
Naive: 9.17319210000278 s
Speedup: 40.998 x
True
```

Lecture 6 - Milestone 2

Find the optimal chunk count for Dask local on your machine. Record n chunks optimal and LIF min in performance notebook (MP2)

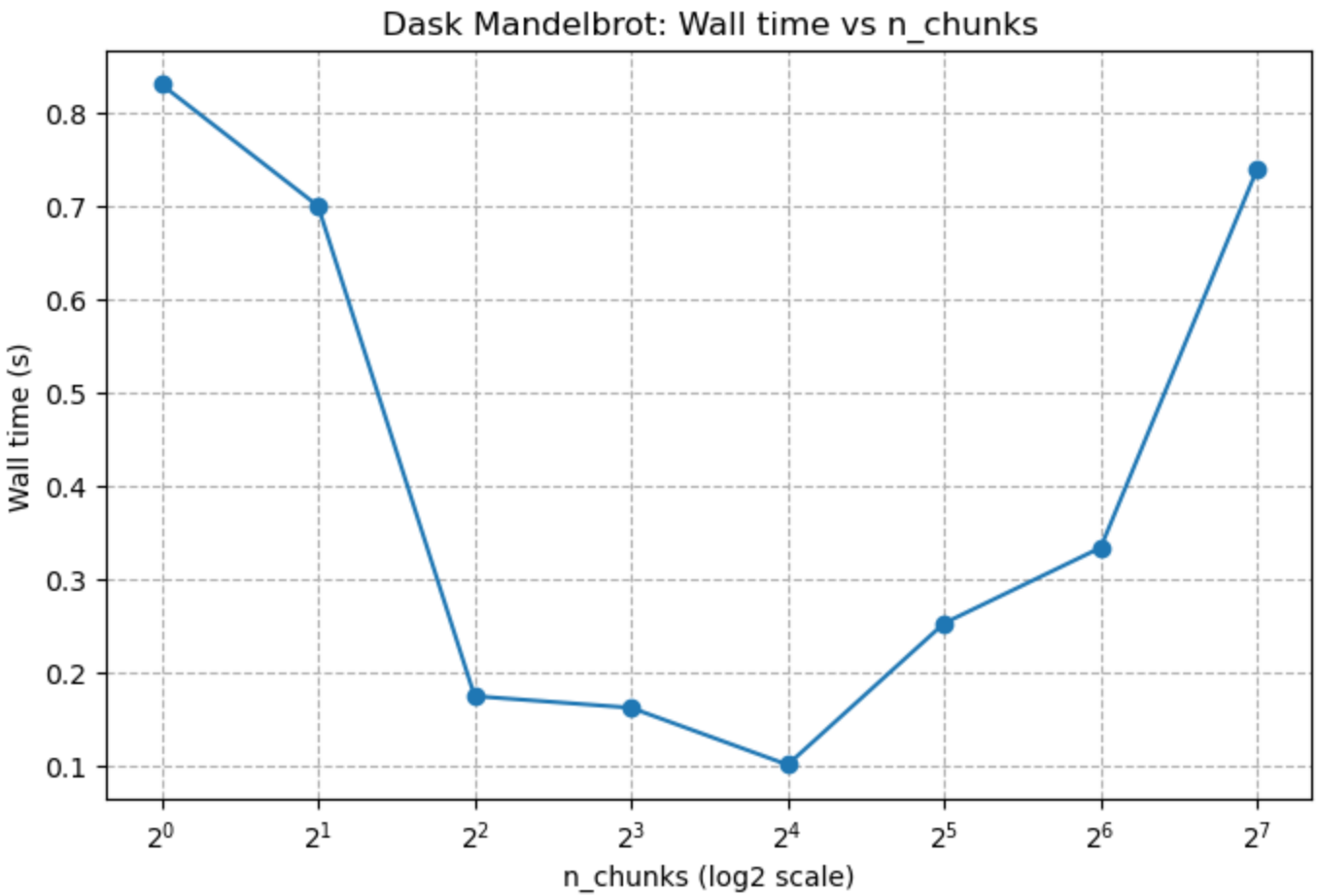
In [8]:

```
from dask import delayed
from dask.distributed import Client, LocalCluster
import dask, time, statistics
import numpy as np
import matplotlib.pyplot as plt

# Sweep over n_chunks
N = 1024
n_runs = 3
x_dim = (-2.0, 1.0)
y_dim = (-1.5, 1.5)
```

```python
max_iter = 100
chunk_values = [1, 2, 4, 8, 16, 32, 64, 128]  # n_chunks to sweep over

cluster = LocalCluster(n_workers=8, threads_per_worker=1, dashboard_address=None)
client = Client(cluster)
client.run(lambda: mb5.mandelbrot_chunk(row_start=0, row_end=8, N=8, x_dim=x_dim, y_dim=y_dim, max_iter=max_iter)) #

results = []

for n_chunks in chunk_values:
    run_times = []
    for _ in range(n_runs):
        start = time.perf_counter()
        mb6.mandelbrot_dask(N, x_dim, y_dim, max_iter, n_chunks=n_chunks)
        run_times.append(time.perf_counter() - start)
    median_time = statistics.median(run_times)
    if n_chunks == 1:
        T1 = median_time  # baseline
        speedup = 1.0
        LIF = 0.0
    else:
        speedup = T1 / median_time
        LIF = n_chunks * median_time / T1 - 1
    results.append((n_chunks, median_time, speedup, LIF))
    print(f"Chunks: {n_chunks:3d} | Time: {median_time:.2f}s | Speedup: {speedup:.2f}x | LIF: {LIF:.2f}")
client.close(); cluster.close()

# Convert results to arrays for plotting
results_array = np.array(results)
n_chunks_arr = results_array[:,0]
times_arr = results_array[:,1]
speedup_arr = results_array[:,2]
LIF_arr = results_array[:,3]

# Plot wall time vs n_chunks
plt.figure(figsize=(8,5))
plt.plot(n_chunks_arr, times_arr, marker='o')
plt.xscale('log', base=2)
plt.xlabel("n_chunks (log2 scale)")
plt.ylabel("Wall time (s)")
plt.title("Dask Mandelbrot: Wall time vs n_chunks")
plt.grid(True, which='both', ls='--')
```

```python
plt.savefig("../figures/dask_chunk_sweep.png", dpi=150)
plt.show()

# Find optimal chunk count
optimal_idx = np.argmin(times_arr)
n_optimal = int(n_chunks_arr[optimal_idx])
t_min_dask = times_arr[optimal_idx]
LIF_min = LIF_arr[optimal_idx]

print(f"Optimal n_chunks: {n_optimal}, t_min: {t_min_dask:.2f}s, LIF_min: {LIF_min:.2f}")
```

```
Chunks:   1 | Time: 0.83s | Speedup: 1.00x | LIF: 0.00
Chunks:   2 | Time: 0.70s | Speedup: 1.19x | LIF: 0.69
Chunks:   4 | Time: 0.18s | Speedup: 4.74x | LIF: -0.16
Chunks:   8 | Time: 0.16s | Speedup: 5.11x | LIF: 0.57
Chunks:  16 | Time: 0.10s | Speedup: 8.19x | LIF: 0.95
Chunks:  32 | Time: 0.25s | Speedup: 3.28x | LIF: 8.75
Chunks:  64 | Time: 0.33s | Speedup: 2.49x | LIF: 24.75
Chunks: 128 | Time: 0.74s | Speedup: 1.12x | LIF: 113.05
```




```
Optimal n_chunks: 16, t_min: 0.10s, LIF_min: 0.95
```

Lecture 6 - Milestone 3

Collect all timings and compare every implementation built so far. Add your Dask local result to the Performance Tracker in Moodle.

In [9]:
```
from dask import delayed
from dask.distributed import Client, LocalCluster
import dask, time, statistics
import numpy as np
from numba import njit
from multiprocessing import Pool
from pathlib import Path
```

```python
# Parameters
warmup = 64
N = 1024
n_runs = 3
x_dim = (-2.0, 1.0)
y_dim = (-1.5, 1.5)
iterations = 100
n_workers = 8
chunk_values = [1, 2, 4, 8, 16, 32, 64, 128]  # sweep over n_chunks
resolution_warmup = (warmup, warmup)
resolution = (N, N)

# 1: Speedup table:
print(f'Speedup Table for resolution=1024:\n',
      f"{'Implementation':>12} | {'Time (s)':>12} | {'Speedup (x)':>12} |")

# Naive Implementation
times = []
for _ in range(n_runs):
    t0 = time.perf_counter()
    mb12.compute_mandelbrot_naive(x_dim=x_dim, y_dim=y_dim, res=resolution)
    times.append(time.perf_counter() - t0)
t_naive = statistics.median(times)
speedup_naive = t_naive/t_naive
print(f"{'Naive Python':>15} | {t_naive:12.3f} | {speedup_naive:12.2f} |")

# Numpy Implementation
times = []
for _ in range(n_runs):
    t0 = time.perf_counter()
    mb12.compute_mandelbrot_numpy(x_dim=x_dim, y_dim=y_dim, res=resolution)
    times.append(time.perf_counter() - t0)
t_numpy = statistics.median(times)
speedup_numpy = t_naive/t_numpy
print(f"{'Numpy':>15} | {t_numpy:12.3f} | {speedup_numpy:12.2f} |")

# Numba Implementation
times = []
_ = mb3.compute_mandelbrot_full(x_dim=x_dim, y_dim=y_dim, res=resolution_warmup)  # warmup JIT
for _ in range(n_runs):
    t0 = time.perf_counter()
```

```
    mb3.compute_mandelbrot_full(x_dim=x_dim, y_dim=y_dim, res=resolution, max_iter=iterations)
    times.append(time.perf_counter() - t0)
t_numba = statistics.median(times)
speedup_numba = t_naive/t_numba
print(f"{'Numba @njit':>15} | {t_numba:12.3f} | {speedup_numba:12.2f} |")

# Parallel Implementation (t_parallel from L5-M2)
index_t_parallel_peak = np.argmin(t_parallel_arr)
t_parallel = t_parallel_arr[index_t_parallel_peak] # optimal t_parallel
speedup_parallel = t_naive/t_parallel
print(f"{'Parallel (opt)':>15} | {t_parallel:12.3f} | {speedup_parallel:12.2f} |")

# Benchmark for Dask implementation (Dask from L6-M2)
speedup_dask = t_naive/t_min_dask
print(f"{'Dask (opt)':>15} | {t_min_dask:12.3f} | {speedup_dask:12.2f} |")
```

```
Speedup Table for resolution=1024:
 Implementation |     Time (s) |  Speedup (x) |
   Naive Python |        8.861 |         1.00 |
          Numpy |        1.830 |         4.84 |
    Numba @njit |        0.074 |       120.47 |
 Parallel (opt) |        0.024 |       361.67 |
     Dask (opt) |        0.102 |        87.26 |
```

Parallel and dask implementation are only computed for the optimal n_chunk which is 64 for parallel (n_chunk=8 * n_workers=8) and 8 for dask.

Section 3: Dask Distributed (Results from: L07-M1)

Lecture 7 - Experiment 1: Chunk size sweep

Fix all worker VMs connected; vary chunk size.

- Record wall time (median of 3 runs) per chunk size
- Note best chunk size
- How does it compare to your L06-M2 result?

Number of workers: 2 Threads per worker: [1, 1] Resolution N=1024 Numba serial: 0.08s Dask best chunk=2: 0.08s | Speedup=1.05x Wall time vs chunks: Chunks: 1 | Time: 0.12s Chunks: 2 | Time: 0.08s Chunks: 4 | Time: 0.08s Chunks: 8 | Time: 0.09s Chunks: 16 | Time: 0.10s Chunks: 32 | Time: 0.13s Chunks: 64 | Time: 0.19s Chunks: 128 | Time: 0.35s Resolution N=2048 Numba serial: 0.33s Dask best chunk=2: 0.21s | Speedup=1.54x Wall time vs chunks: Chunks: 1 | Time: 0.39s Chunks: 2 | Time: 0.21s Chunks: 4 |

Time: 0.22s Chunks: 8 | Time: 0.24s Chunks: 16 | Time: 0.24s Chunks: 32 | Time: 0.27s Chunks: 64 | Time: 0.30s Chunks: 128 | Time: 0.42s Resolution N=4096 Numba serial: 1.31s Dask best chunk=2: 0.81s | Speedup=1.62x Wall time vs chunks: Chunks: 1 | Time: 1.41s Chunks: 2 | Time: 0.81s Chunks: 4 | Time: 0.83s Chunks: 8 | Time: 0.88s Chunks: 16 | Time: 0.83s Chunks: 32 | Time: 0.88s Chunks: 64 | Time: 0.89s Chunks: 128 | Time: 0.94s Resolution N=8192 Numba serial: 5.23s Dask best chunk=4: 3.14s | Speedup=1.67x Wall time vs chunks: Chunks: 1 | Time: 5.55s Chunks: 2 | Time: 3.18s Chunks: 4 | Time: 3.14s Chunks: 8 | Time: 3.21s Chunks: 16 | Time: 3.44s Chunks: 32 | Time: 3.20s Chunks: 64 | Time: 3.18s Chunks: 128 | Time: 3.17s

Experiment 2: Worker scaling Fix best chunk size; vary number of active worker VMs.

---------- 2 Workers ---------- Number of workers: 2 Threads per worker: [1, 1] Resolution N=1024 Optimal n_chunk: 2 Numba time: 0.08s Dask time: 0.08s Speedup: 1.01x Resolution N=2048 Optimal n_chunk: 2 Numba time: 0.33s Dask time: 0.21s Speedup: 1.55x Resolution N=4096 Optimal n_chunk: 2 Numba time: 1.33s Dask time: 0.84s Speedup: 1.58x Resolution N=8192 Optimal n_chunk: 4 Numba time: 5.26s Dask time: 3.25s Speedup: 1.62x ---------- 4 Workers ---------- Number of workers: 4 Threads per worker: [1, 1, 1, 1] Resolution N=1024 Optimal n_chunk: 2 Numba time: 0.08s Dask time: 0.08s Speedup: 0.99x Resolution N=2048 Optimal n_chunk: 2 Numba time: 0.33s Dask time: 0.24s Speedup: 1.36x Resolution N=4096 Optimal n_chunk: 2 Numba time: 1.32s Dask time: 0.86s Speedup: 1.54x Resolution N=8192 Optimal n_chunk: 4 Numba time: 5.27s Dask time: 3.00s Speedup: 1.76x

Final Performance Table

Performance table includes all six implementations with runtimes, worker/core counts, and speedups

In [10]:
```
# Collect all results in a table: (Implementation, Time(s), Workers/Cores, Speedup vs Naive)
# Resolution = 1024
performance_data_1024 = [
    ("Naive Python 1024", t_naive, 1, t_naive / t_naive),
    ("Numpy 1024", t_numpy, 1, t_naive / t_numpy),
    ("Numba 1024", t_numba, 1, t_naive / t_numba),
    ("Parallel 1024", t_parallel, 8, t_naive / t_parallel),
    ("Dask local 1024", t_min_dask, 8, t_naive / t_min_dask),
    ("Dask distributed 1024", 0.08, 2, t_naive / 0.08),
    ("Dask distributed 1024", 0.08, 4, t_naive / 0.08)
    ]
# Print header
print(f"{'Implementation (1024)':<23} | {'Time (s)':>10} | {'Workers/Cores':>14} | {'Speedup':>8}")
print("-"*66)
# Print each row
for name, time_val, cores, speedup in performance_data_1024:
    print(f"{name:<23} | {time_val:10.2f} | {cores:14d} | {speedup:8.2f}")

# Resolution = 2048
performance_data_2048 = [
    ("Numba 2048", 0.33, 1, 0.33 / 0.33),
    ("Dask distributed 2048", 0.21, 2, 0.33 / 0.21),
    ("Dask distributed 2048", 0.24, 4, 0.33 / 0.24)]
# Print header
```

```python
print()
print(f"{'Implementation (2048)':<23} | {'Time (s)':>10} | {'Workers/Cores':>14} | {'Speedup':>8}")
print("-"*66)
# Print each row
for name, time_val, cores, speedup in performance_data_2048:
    print(f"{name:<23} | {time_val:10.2f} | {cores:14d} | {speedup:8.2f}")

# Resolution = 4096
performance_data_4096 = [
    ("Numba 4096", 1.31, 1, 1.31 / 1.31),
    ("Dask distributed 4096", 0.84, 2, 1.31 / 0.84),
    ("Dask distributed 4096", 0.86, 4, 1.31 / 0.86)]
# Print header
print()
print(f"{'Implementation (4096)':<23} | {'Time (s)':>10} | {'Workers/Cores':>14} | {'Speedup':>8}")
print("-"*66)
# Print each row
for name, time_val, cores, speedup in performance_data_4096:
    print(f"{name:<23} | {time_val:10.2f} | {cores:14d} | {speedup:8.2f}")

# Resolution = 8192
performance_data_8192 = [
    ("Numba 8192", 5.23, 1, 5.23 / 5.23),
    ("Dask distributed 8192", 3.25, 2, 5.23 / 3.25),
    ("Dask distributed 8192", 3.00, 4, 5.23 / 3.00)]
# Print header
print()
print(f"{'Implementation (8192)':<23} | {'Time (s)':>10} | {'Workers/Cores':>14} | {'Speedup':>8}")
print("-"*66)
# Print each row
for name, time_val, cores, speedup in performance_data_8192:
    print(f"{name:<23} | {time_val:10.2f} | {cores:14d} | {speedup:8.2f}")
```

Implementation (1024)	Time (s)	Workers/Cores	Speedup
Naive Python 1024	8.86	1	1.00
Numpy 1024	1.83	1	4.84
Numba 1024	0.07	1	120.47
Parallel 1024	0.02	8	361.67
Dask local 1024	0.10	8	87.26
Dask distributed 1024	0.08	2	110.76
Dask distributed 1024	0.08	4	110.76

Implementation (2048)	Time (s)	Workers/Cores	Speedup
Numba 2048	0.33	1	1.00
Dask distributed 2048	0.21	2	1.57
Dask distributed 2048	0.24	4	1.38

Implementation (4096)	Time (s)	Workers/Cores	Speedup
Numba 4096	1.31	1	1.00
Dask distributed 4096	0.84	2	1.56
Dask distributed 4096	0.86	4	1.52

Implementation (8192)	Time (s)	Workers/Cores	Speedup
Numba 8192	5.23	1	1.00
Dask distributed 8192	3.25	2	1.61
Dask distributed 8192	3.00	4	1.74